ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Сучасні каркаси web-додатків

Лісняк Андрій Олександрович

Лабораторна робота 4. Створення вкладок із використанням Slots, Provide/Inject, dynamic components і KeepAlive

Мета

Навчитися створювати у новому Vue-застосунку складений компонент вкладок, частини якого обмінюються локальним контекстом через provide/inject, відображають панелі слотами й динамічними компонентами та зберігають їхній локальний стан за допомогою KeepAlive.

Теоретичні відомості

Складений компонент і межі відповідальності

У компонентній архітектурі прості елементи інтерфейсу — кнопки, поля введення чи інформаційні картки — проєктують як самостійні компоненти, що отримують усі необхідні дані через вхідні пропси.

Проте в застосунках часто виникають складніші структури, утворені кількома взаємопов’язаними частинами, які ділять спільний стан і повинні діяти узгоджено. Спроба реалізувати таку структуру одним монолітним компонентом швидко вичерпує себе. Що більше налаштувань вимагає кожна внутрішня деталь, то більше конфігураційних пропсів доводиться передавати. У результаті замість наочної декларативної розмітки розробник змушений описувати інтерфейс складними масивами об’єктів у JavaScript, а сам компонент стає перевантаженим і негнучким.

Типовими прикладами таких структур є система вкладок або багатоступенева форма. Наприклад, у системі вкладок спільний стан активного розділу координує смугу кнопок навігації та відповідні панелі, де кожна панель має власну розмітку й локальні дані.

Щоб подолати цю проблему, такі інтерфейси будують як систему невеликих взаємодіючих компонентів, що утворюють складений компонент (compound component).

Складений компонент (compound component) — кілька окремих компонентів, які спільно керують одним станом і використовуються лише разом.

Контейнер такого об’єднання володіє станом і координує спільну поведінку, тоді як дочірні елементи описують окремі частини інтерфейсу. Заздалегідь контейнер не знає ні точної кількості частин, ні їхнього порядку: склад визначає той шаблон, у якому цей компонент викликається.

Усі подальші інженерні рішення та механізми розглядаються на прикладі системи вкладок `Tabs`.

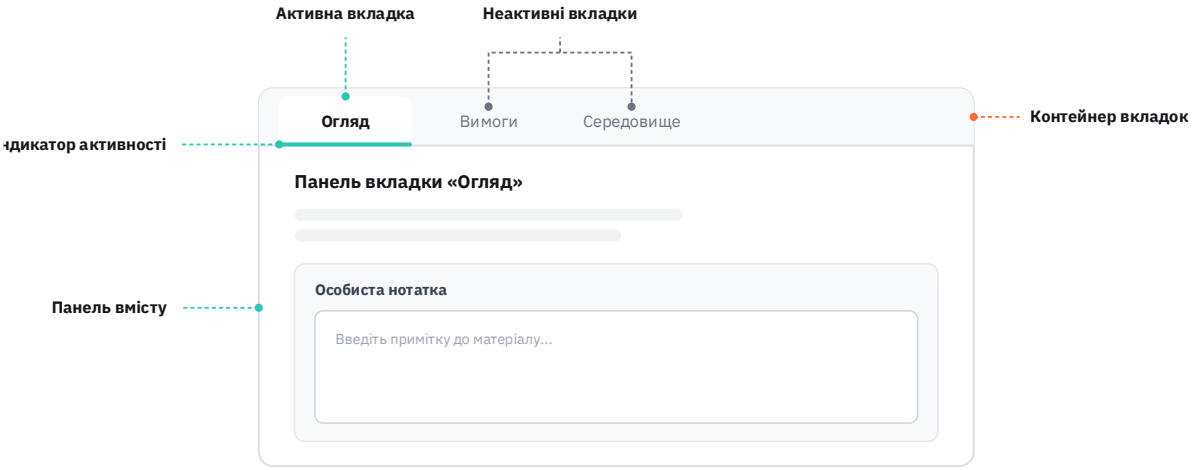


Рисунок 1 — Складений компонент Tabs.

Для реалізації такого інтерфейсу систему вкладок поділяють на два взаємопов'язані компоненти: батьківський контейнер `Tabs` та дочірні декларації `Tab`. Контейнер володіє спільним станом і виконує відображення навігації, тоді як кожна декларація описує окрему вкладку та надає вміст її панелі. Структуру такого поділу наведено на рисунку 2.

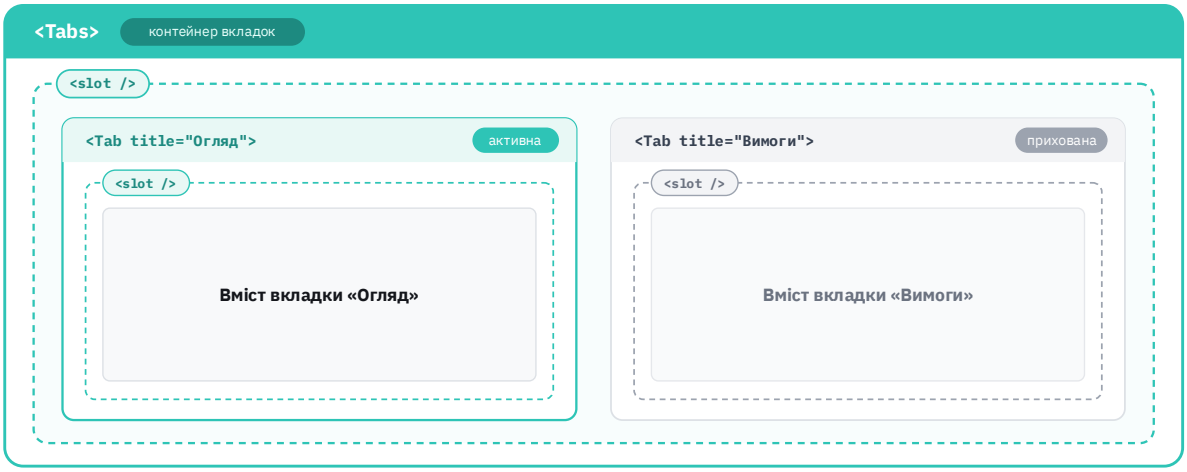


Рисунок 2 — Розподіл системи вкладок на компоненти Tabs та Tab.

У коді це має наступний вигляд:

```
<!-- Декларативний опис складу вкладок у шаблоні батьківського
компонента -->
<Tabs v-model="activeTab" variant="underline">
  <Tab slug="overview" title="Огляд">
    <p>Вміст вкладки «Огляд»</p>
  </Tab>
  <Tab slug="requirements" title="Вимоги">
    <p>Вміст вкладки «Вимоги»</p>
  </Tab>
</Tabs>
```

Декларація `Tab` — теж оболонка і вміст вкладок лежить у його власному слоті за замовчуванням. Оскільки цей вміст належить шаблону батьківського компонента, усі потрібні змінні зв'язуються просто на місці, і `Tab` нічого не пересилає далі. Власний стан декларація може віддавати цьому ж вмісту пропсами слота через запис `<slot :is-active="isActive"/>`, що дає вмісту знати, чи активна зараз ця вкладка, не роблячи його обізнаним про весь контекст вкладок.

Передавання контексту через provide/inject

Поділ системи на зовнішній контейнер `Tabs` та внутрішні декларації `Tab` створює зрозумілий шаблон, проте породжує проблему прокидання даних. Контейнер `Tabs` рендерить кнопки навігації, але не знає наперед, які саме вкладки оголошено в його слоті. Водночас кожна вкладка `Tab` повинна знати, чи активна вона в поточний момент, щоб показати або приховати свій вміст.

Звичайне передавання через пропси змусило б розробника вручну зв'язувати стан кожної вкладки в батьківському шаблоні. Якщо ж між контейнером і вкладками з'являються проміжні вузли (розмітка, стилістичні обгортки чи сітка), кожен проміжний рівень змушений приймати й передавати чужі параметри. Цей антипатерн називають **прокиданням пропсів (prop drilling)**.

Впровадження залежностей (provide / inject) — механізм Vue для передавання даних і методів від компонента-предка до його нащадків на будь-яку глибину дерева без проміжних пропсів.

Ключ ін'єкції (injection key) — значення, за яким нащадок знаходить наданий предком контекст.

Рядковий ключ у великому застосунку легко збігається з чужим, тому ключі створюють як `Symbol` і експортують з окремого модуля: один експортований символ гарантує, що `provide` та `inject` посилаються на те саме.

```
// tabsKey.js
export const tabsKey = Symbol('tabs')
```

Порівняння прокидання пропсів крізь проміжні рівні та прямого зв'язку через `provide / inject` наведено на рисунку 3.

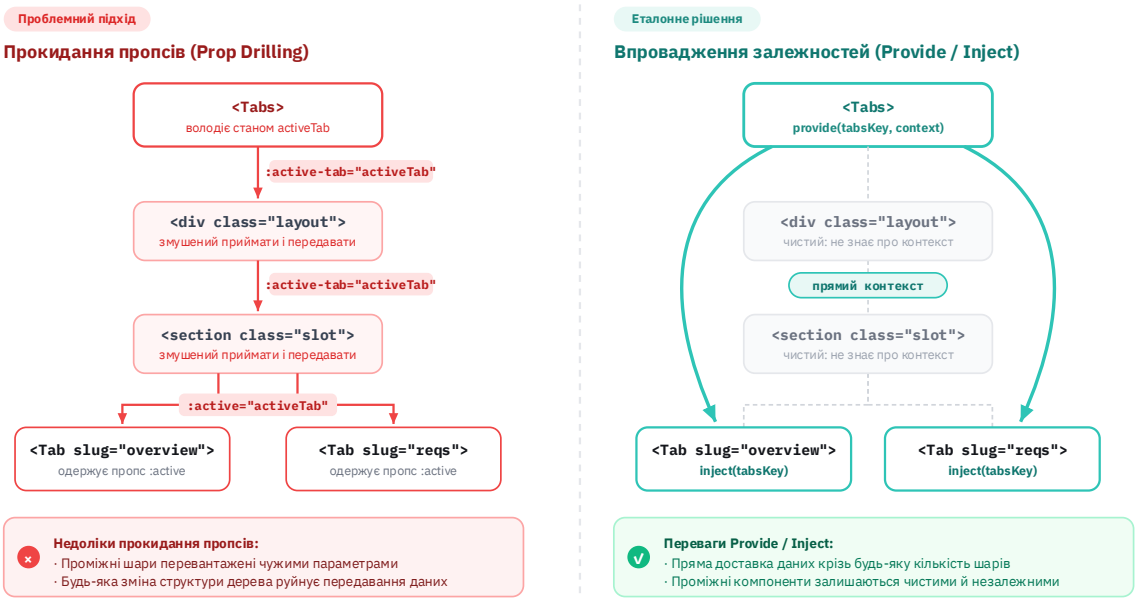


Рисунок 3 — Порівняння передавання даних через пропси (Prop Drilling) та через Provide / Inject.

Список вкладок контейнер отримує не з пропсів, а від самих декларацій. У хуку `onMounted` кожна декларація додає свої дані до спільного масиву через `registerTab`, а в `onBeforeUnmount` вилучає їх звідти через `unregisterTab`. Завдяки цьому вкладка, прибрана з шаблону директивою `v-if`, зникає і зі списку кнопок, а контейнер за потреби переводить вибір на першу доступну вкладку. Кожен екземпляр контейнера виконує власний виклик `provide`, а `inject` у нащадка знаходить найближчого предка, який надав цей ключ. Тому два набори вкладок на одній сторінці мають незалежні контексти, і перемикання в одному з них не змінює активну вкладку в іншому.

Панель розміткою або значенням і кешування її стану

Вміст панелі потрапляє до декларації двома шляхами, і вибір між ними є усвідомленим архітектурним рішенням.

Перший шлях — розмітка у слоті. Він доречний, коли склад вкладок відомий під час верстки шаблону. Кожну панель видно за іменем, її параметри зв'язуються поруч у тому ж контексті, і жодне значення не мандрує крізь декларацію `Tab`. Другий шлях потрібен тоді, коли набір вкладок формується динамічно на основі даних — масиву налаштувань, прав доступу чи відповіді сервера. Статичної розмітки для кожної панелі в такому разі не існує, тому компонент передають значенням.

Динамічний компонент — вбудований елемент `<component>`, який відтворює компонент, переданий атрибутом `is`.

Значенням `is` є посилання на об'єкт компонента або його зареєстроване ім'я. Разом із компонентом передають його власні параметри через `v-bind` без аргументу: запис `<component :is="panel" v-bind="panelProps"/>` розгортає об'єкт властивостей і передає їх як звичайні пропси. Це є компромісом другого

шляху: параметри панелі змушені проходити крізь пропси декларації `Tab` (`panel` та `panelProps`), якими сама декларація не користується.

Правило вибору однозначне — якщо склад вкладок статично описано в шаблоні, панель передають розміткою; якщо склад приходить даними, панель передають значенням.

Обидва шляхи вимагають рішення щодо життєвого циклу неактивної панелі. Коли користувач обирає іншу вкладку, неактивний вміст необхідно приховати. Порівняння трьох можливих способів наведено в таблиці 1.

Таблиця 1 — Порівняння способів приховування та збереження стану панелей

Підхід	Стан у DOM	Збереження даних і форми	Життєвий цикл компонента	Рекомендоване застосування
`v-show`	Залишається в DOM (`display: none`)	Зберігається автоматично	Компонент не знає про приховування; фонові процеси тривають	Кілька простих статичних панелей
`v-if`	Повністю вилучається (демонтаж)	Втрачається (стан скидається до початкового)	Викликаються `onBeforeUnmount` та `onUnmounted`	Панелі без власного локального стану
`<KeepAlive>`	Вилучається з DOM	Зберігається в кеші оперативної пам'яті	Викликаються хуки `onDeactivated` та `onActivated`	Форми, фільтри, компоненти з важким станом

Вбудований компонент KeepAlive — компонент Vue, який зберігає екземпляри вилучених із DOM нащадків у кеші пам'яті замість їхнього знищення.

Екземпляр, що входить до кешованого дерева, під час вилучення з DOM переходить у деактивований стан, а під час повторного вставлення активується. Прямим нащадком `<KeepAlive>` може бути лише один активний екземпляр компонента, тому у слот декларації кладуть один компонент панелі. Кількість збережених екземплярів обмежує пропс `max`: із ним `KeepAlive` діє як LRU-кеш (Least Recently Used) і знищує найдавніше використаний екземпляр, коли межу перевищено.

Активація (onActivated) — хук життєвого циклу, який викликається під час початкового монтування і щоразу після повернення екземпляра з кешу на екран.

Деактивація (onDeactivated) — хук життєвого циклу, який викликається під час переходу екземпляра з DOM до кешу, а також під час його остаточного демонтажу.

Ці хуки дозволяють зупиняти ресурсомісткі процеси — таймери, слухачі подій, анімації чи мережеві з'єднання — доки панель прихована, і відновлювати їх після повернення користувача.

Універсальний шаблон компонента `Tab.vue` об'єднує підтримку слота, динамічного компонента та кешування:

```
<!-- Tab.vue: підтримка розмітки у слоті або динамічного компонента
з кешуванням -->
<section v-show="isActive" class="tabs__panel" role="tabpanel">
  <KeepAlive>
    <slot v-if="isActive && $slots.default" :is-active="isActive"/>
    <component v-else-if="isActive" :is="panel" v-
bind="panelProps"/>
  </KeepAlive>
</section>
```

Слот має безумовний пріоритет: якщо декларація одержала вміст через `$slots.default`, пропс `panel` не розглядається. Завдяки цьому один компонент обслуговує обидва сценарії використання.

Функційний опис Tabs і Tab за IDEF0

IDEF0 подає компонент як функцію з чотирма типами стрілок: **вхід** (Input) — дані для опрацювання, **керування** (Control) — правила й обмеження, **вихід** (Output) — результат, **механізм** (Mechanism) — засоби виконання. Входи розміщують ліворуч, керування — згори, виходи — праворуч, механізми — знизу.

A1 «Керувати вкладками» (`Tabs`). Компонент отримує `modelValue`, декларації `Tab` і команди користувача; працює за контрактом `v-model`, значенням `variant` і правилами WAI-ARIA; формує `update:modelValue`, смугу `tablist` та контекст `tabsKey`. Реалізацію забезпечують Vue 3, `provide/inject` і реактивний реєстр.



Рисунок 4 — Вузол A1 «Керувати вкладками» за IDEF0. Джерело: схема автора курсу за ISO/IEC/IEEE 31320-1:2012.

A2 «Показати панель» (`Tab`). Компонент отримує `slug`, `title`, `disabled` і вміст через слот або пропси `panel` та `panelProps`; працює за контекстом `tabsKey` і правилами WAI-ARIA; реєструє вкладку, надає `isActive` і показує активну панель. Реалізацію забезпечують lifecycle-хуки, `<component :is>` і `KeepAlive`.



Рисунок 5 — Вузол A2 «Показати панель» за IDEF0. Джерело: схема автора курсу за ISO/IEC/IEEE 31320-1:2012.

Доступність і клавіатурна навігація за WAI-ARIA

Згівдно до ARIA специфікації структурні елементи вкладок повинні мати наступні атрибути:

- **список вкладок (role="tablist")** — контейнер, який групує кнопки перемикання панелей;
- **dкладка (role="tab")** — елемент керування, що обирає та відображає пов’язану з ним панель;
- **ganель вкладки (role="tabpanel")** — контейнер вмісту, пов’язаного з окремою вкладкою.

Роль `tablist` позначає групу вкладок і потребує доступної назви через `aria-label` або `aria-labelledby`. Кнопка вкладки посилається на свою панель через `aria-controls`, а панель на кнопку — через `aria-labelledby`. Вибрана кнопка має `aria-selected="true"`, решта — `aria-selected="false"`. Активна панель відображається, а панелі інших вкладок приховуються. `tabindex="0"` на панелі дає змогу перейти до початку її вмісту клавішею `Tab`.

Патерн Roving Tabindex — спосіб керування фокусом, за якого лише вибрана вкладка входить у послідовність переходу клавішею `Tab`; інші вкладки мають `tabindex="-1"`.

У цій роботі використовується **автоматична активація**: щойно вкладка отримує фокус, вона стає вибраною, а пов’язана з нею панель відображається. Навігація має працювати так:

1. **Вхід до списку.** Натискання `Tab` перед списком вкладок установлює фокус на активній вкладці. Неактивні вкладки при цьому пропускаються у звичайній послідовності переходу.
2. **Стрілка вправо.** `ArrowRight` переносить фокус на наступну вкладку й одразу активує її. Після останньої вкладки фокус переходить на першу.
3. **Стрілка вліво.** `ArrowLeft` переносить фокус на попередню вкладку й одразу активує її. Перед першою вкладкою вибирається остання.
4. **Початок і кінець списку.** `Home` переносить фокус на першу вкладку та активує її, а `End` — на останню.
5. **Вихід зі списку.** Натискання `Tab` на вкладці не перебирає решту кнопок, а переносить фокус до активної панелі. Наступне натискання продовжує рух елементами панелі або сторінки. `Shift+Tab` рухає фокус у зворотному напрямку.

Після натискання стрілок, `Home` або `End` додатково натискати `Enter` чи `Space` не потрібно. Автоматичну активацію застосовують лише тоді, коли нову панель можна показати без помітної затримки.

У реалізації цієї лабораторної недоступні вкладки пропускаються під час переміщення стрілками й не отримують фокуса.

Функціональні вимоги

- У новому проєкті на одній сторінці одночасно показано три незалежні набори вкладок із варіантами underline, pills і boxed; кожен набір містить щонайменше три вкладки
- У кожному наборі активною є рівно одна вкладка; її зміна показує відповідну панель без перезавантаження сторінки
- Кожен набір має власний активний стан; перемикання одного набору не змінює два інші
- У кожному наборі щонайменше одна панель містить локальний стан, наприклад текстове поле, прапорець або лічильник; після переходу на іншу вкладку й повернення введені значення зберігаються
- Один набір отримує панелі розміткою через слоти Tab, другий формується з масиву та передає панелі як значення компонента
- У панелі одного з наборів використано ModalDialog із лабораторної роботи 3; діалог відкривається і закривається без втрати локального стану панелі
- Вкладки мають автоматичну активацію; Tab установлює фокус на активній вкладці, ArrowLeft, ArrowRight, Home і End переміщують фокус та одразу перемикають панель, а наступний Tab переходить до активної панелі

- За ширин 360×800, 768×1024, 1440×900 і 844×390 усі набори вкладок і діалог доступні без горизонтальної прокрутки сторінки й обрізаного тексту
- Сторінка містить видимі ПІБ, групу й відомості про середовище для ідентифікації знімків екрана

Технічні вимоги

- Проєкт lab4 створено з нуля через Vite; файли й конфігурацію попереднього проєкту цілком не копійовано
- Використано Vue 3 SFC, JavaScript, Composition API та script setup; готові бібліотеки вкладок, компонентів і кешування не використовуються
- Tabs має пропси modelValue і variant та подію update:modelValue; Tab має пропси slug і title, а панель одержує власним слотом за замовчуванням або пропсами panel і panelProps, де слот має перевагу
- Tabs відтворює default slot для декларацій Tab і надає окремий реактивний контекст кожному своєму екземпляру
- Контекст винесено в tabsKey.js на основі Symbol та містить readonly activeSlug, idPrefix, setActiveTab, registerTab і unregisterTab; Tab отримує його через inject
- Tab реєструється під час монтування, знімає реєстрацію перед демонтажем і повідомляє зрозумілу помилку за відсутності Tabs або повторення slug
- Щонайменше один набір задає панелі слотами, а інший — масивом через <component :is>; у KeepAlive завжди рівно один дочірній компонент, який демонтується умовою v-if, а не прихований лише v-show
- Ознаку активності панель одержує пропсом слота; контекст вкладок вона не читає
- Кнопки та панелі мають role=tab, role=tablist і role=tabpanel, пов’язані унікальними id через aria-controls та aria-labelledby; список вкладок має доступну назву, а активність передано через aria-selected і tabindex
- Стилі компонентів розміщено у style scoped lang=scss, названо за BEM і сформовано через Tailwind @apply; у шаблонах немає повторюваних довгих наборів utility-класів
- Кольори й шрифти задано ролями з конфігурації зразка оформлення; власних значень кольору в компонентах немає
- Tailwind CSS 3.4.17 підключено локально через Vite/PostCSS, Sass встановлено як devDependency, а package-lock.json збережено
- Варіанти underline, pills і boxed задано BEM-модифікаторами; стилі не створюються конкатенацією назв класів
- Контракти Tabs і Tab описано як функційні вузли IDEF0 з розподілом інтерфейсів на Input, Control, Output і Mechanism

- Дані зберігаються в пам’яті; HTTP-запити, маршрутизація, localStorage та глобальний стан не використовуються

Необхідне обладнання та матеріали

- Node.js 24 LTS, npm, Git і редактор коду
- Власний компонент ModalDialog із лабораторної роботи 3
- Tailwind CSS 3.4.17, PostCSS 8, Autoprefixer 10 і Sass
- Зразок оформлення вкладок із репозиторію znu-lab-starters
- Браузер із DevTools та режимом емуляції розмірів екрана

Хід роботи

У денній і заочній формах створюється новий Vue-проєкт у каталозі `lab4`. Проєкт попередньої роботи цілком не копіюється. З нього повторно використовується лише власний компонент `ModalDialog`.

1. Створення нового проєкту

У корені власного репозиторію створіть застосунок з шаблону Vite:

```
npm create vite@latest lab4 -- --template vue
cd lab4
npm install
npm install -D tailwindcss@3.4.17 postcss@8 autoprefixer@10 sass
npx tailwindcss init -p
```

У `tailwind.config.js` додайте до `content` файл `index.html` і всі файли `vue` та `js` у каталозі `src`. У головному файлі стилів підключіть шари `@tailwind base`, `@tailwind components` і `@tailwind utilities`.

Запустіть `npm run dev -- --port 5173 --strictPort` і переконайтеся, що стартова сторінка відкривається без помилок. Перенесіть до нового проєкту власний `ModalDialog.vue` з лабораторної роботи 3 разом із файлами, від яких він безпосередньо залежить. Інші компоненти попереднього проєкту не копіюйте.

2. Підготовка панелей вмісту

Для повноцінної перевірки збереження локального стану підготуйте окремі компоненти панелей. Створіть у каталозі `src/components/demo-panels/` три різні панелі:

- `OverviewPanel.vue` — панель із полем введення нотатки (`v-model="note"`), де користувач може ввести довільний текст;
- `RequirementsPanel.vue` — панель зі списком вимог, інтерактивними прапорцями та кнопковим лічильником виконаних пунктів;
- `EnvironmentPanel.vue` — відомості про середовище розробки та кнопка відкриття перенесеного діалогового вікна `ModalDialog`.

Кожна панель має містити власний реактивний стан, зміна якого добре помітна під час тестування перемикання вкладок.

3. Контракти Tabs і Tab

Вхідні пропси `Tabs`:

Пропс	Тип	За замовчуванням	Призначення
`modelValue`	`String`	обов’язковий	активна вкладка
`variant`	`String`	`'underline'`	оформлення смуги

Вихідні події `Tabs`:

Подія	Навантаження	Коли надсилається
`update:modelValue`	`slug` нової активної вкладки	вибір кнопки мишею або клавіатурою

Вхідні пропси `Tab`:

Пропс	Тип	За замовчуванням	Призначення
`slug`	`String`	обов’язковий	унікальний ідентифікатор
`title`	`String`	обов’язковий	напис на кнопці
`disabled`	`Boolean`	`false`	вкладку не можна обрати
`panel`	`Object`	`null`	панель значенням
`panelProps`	`Object`	`{}`	параметри панелі

Пропс `panel` потрібен лише тоді, коли панель не передано слотом. Власних подій `Tab` не має: активну вкладку він змінює викликом `setActiveTab` з контексту. Слот за замовчуванням приймає компонент панелі й одержує пропс слота `isActive`.

Опишіть реалізацію відповідно до моделей ICOM для вузла `A1 «Керувати вкладками»` та вузла `A2 «Показати панель»`. Звірте розподіл входів, керування, виходів і механізмів із рисунками 4 та 5 теоретичних відомостей.

4. Контекст вкладок

Створіть `src/components/tabs/tabsKey.js`:

```
export const tabsKey = Symbol('tabs')
```

Під цим ключем кожен екземпляр `Tabs` надає власний об’єкт контексту такого складу:

```
class TabsContext {
    activeSlug // readonly ref зі slug активної вкладки
    idPrefix // унікальний префікс ідентифікаторів набору
    setActiveTab(slug)// робить вкладку активною
    registerTab({slug, title, disabled}) // додає вкладку до списку
    unregisterTab(slug) // вилучає вкладку зі списку
}
```

Префікс `idPrefix` потрібен обом компонентам: за ним `Tabs` будує `id` кнопок, а `Tab` — `id` своєї панелі, тож посилання `aria-controls` і `aria-labelledby` сходяться без спільного глобального лічильника.

Запис описує контракт; у `Tabs.vue` цей об'єкт збирають у `setup` зі звичайних `ref` і функцій та передають викликом `provide(tabsKey, context)`. Окремого класу писати не потрібно.

У `Tab.vue` одержіть контекст через `inject(tabsKey)`, зареєструйте вкладку в `onMounted` і зніміть реєстрацію в `onBeforeUnmount`. Поза `Tabs` компонент зупиняється з помилкою, повторний `slug` — теж. Якщо активну вкладку видалено, `Tabs` обирає першу доступну.

5. Розмітка та SCSS-стилі зі зразка оформлення

Готовий вигляд однієї смуги вкладок разом із назвами класів та атрибутами WAI-ARIA наведено у зразку оформлення у репозиторії znu-lab-starters (файл `TabsDesign.vue`). Там немає логіки перемикання чи контексту вкладок — лише розмітка за стандартом WAI-ARIA та стилі базового варіанта `underline`.

Розділіть розмітку та правила між двома новими компонентами:

- `Tabs.vue` отримує кореневий блок `.tabs`, навігаційну смугу `.tabs__list`, кнопки `.tabs__tab` з модифікатором активного стану `.tabs__tab--active`, а також модифікатори варіантів оформлення (`tabs--underline`, `tabs--pills`, `tabs--boxed`);
- `Tab.vue` отримує контейнер панелі `.tabs__panel`.

Стилі компонентів розміщуйте у `<style scoped lang="scss">` з BEM-назвами, а повторювані набори утиліт виносьте через `@apply`.

6. Контейнер Tabs: навігація й доступність

У шаблоні `Tabs.vue` розмістіть смугу `role="tablist"` і згенеруйте кнопки за масивом зареєстрованих вкладок. Безпосередньо під смугою відтворіть default slot `<slot />`, у якому дочірні `Tab` рендеритимуть свої панелі.

Кожному екземпляру `Tabs` надайте унікальний префікс ідентифікаторів (наприклад, за допомогою функції `useId()`) і передайте його полем `idPrefix` контексту, щоб `Tab` будував `id` панелі за тим самим правилом. Дайте списку доступну назву `aria-label`, проставте ролі та взаємні посилання `aria-controls` і `aria-labelledby`.

Реалізуйте автоматичну активацію та патерн Roving Tabindex:

- активна кнопка отримує `aria-selected="true"` і `tabindex="0"`, інші кнопки — `aria-selected="false"` і `tabindex="-1"`;

- клавіші `ArrowLeft`, `ArrowRight`, `Home` і `End` переміщують фокус і одразу показують панель відповідної вкладки;
- стрілки здійснюють циклічний перехід між першою та останньою вкладками;
- вимкнені вкладки (`disabled`) пропускаються під час перебору клавіатурою і не отримують фокуса;
- клавіша `Tab` переводить фокус на активну вкладку, а наступне натискання `Tab` виводить фокус до активної панелі або її внутрішніх елементів.

7. Компонент Tab: слоти, динамічні компоненти та KeepAlive

У `Tab.vue` забезпечте підтримку двох способів передавання панелі: через слот за замовчуванням або пропсами `panel` і `panelProps`:

```
<section
  :id="`${idPrefix}-panel-${slug}`"
  v-show="isActive"
  class="tabs__panel"
  role="tabpanel"
  :aria-labelledby="`${idPrefix}-tab-${slug}`"
  tabindex="0"
>
  <KeepAlive>
    <slot v-if="isActive && $slots.default" :is-active="isActive"/>
    <component v-else-if="isActive" :is="panel" v-
bind="panelProps"/>
  </KeepAlive>
</section>
```

Атрибут `tabindex="0"` ставить панель у послідовність обходу клавішею `Tab` одразу після активної вкладки, а `aria-labelledby` називає її тією самою кнопкою, яка посилається на панель через `aria-controls`.

Слот має безумовний пріоритет над пропсом `panel`. Усередині `<KeepAlive>` завжди перебуває рівно один дочірній компонент, який демонтується умовою `v-if`, коли вкладка неактивна. Не замінюйте цей механізм лише директивою `v-show`: прихована панель залишається змонтованою в DOM і не відтворює справжнє кешування стану.

8. Компонування сторінки та трьох варіантів вкладок

Очистьте стартовий шаблон Vite. Угорі `App.vue` розмістіть блок із видимими даними студента (прізвище, ініціали, група) та відомостями про середовище розробки для однозначної ідентифікації знімків екрана.

Нижче підключіть три незалежні набори вкладок:

1. `underline` — основний набір, у якому панелі передаються компонентами через слоти декларацій `Tab` (`OverviewPanel`, `RequirementsPanel`, `EnvironmentPanel`);

2. `pills` — набір, сформований директивою `v-for` за конфігураційним масивом об’єктів із полями `slug`, `title`, посиланням на компонент `panel` та об'єктом `panelProps`;
3. `boxed` — набір із вимкненою вкладкою (`disabled`) та панеллю, на якій розташована кнопка виклику `ModalDialog`; окремого компонента для неї не створюйте — підключіть удруге вже наявний `EnvironmentPanel`. Два екземпляри одного компонента мають зберігати свій стан незалежно один від одного.

Зв’яжіть кожен екземпляр з окремою змінною `ref(...)` через `v-model`. Перемикання вкладок у будь-якому наборі не повинно змінювати стан двох інших. Переконайтеся, що в кожному варіанті хоча б одна панель має змінений локальний стан, який зберігається після перемикання.

9. Перевірка та здача

Складіть таблицю сценаріїв тестування й для кожного зафіксуйте очікуваний та фактичний результат. Обов’язково перевірте:

- одночасну присутність трьох варіантів оформлення та незалежність їхніх станів;
- перемикання мишею та клавіатурою (`ArrowLeft`, `ArrowRight`, `Home`, `End`);
- циклічний перехід стрілками та пропуск вимкненої вкладки;
- збереження введеного тексту, прапорців і лічильника в панелях після переходу на іншу вкладку та повернення назад;
- відкриття `ModalDialog` із панелі та збереження її стану після закриття вікна;
- адаптивність інтерфейсу без горизонтальної прокрутки сторінки на розмірах 360×800, 768×1024, 1440×900 і 844×390;
- збереження екземпляра компонента в кеші `<KeepAlive>` через Vue DevTools.

Зробіть знімок сторінки з трьома наборами вкладок. Окремо зробіть парні знімки панелі зі зміненими даними до перемикання та після повернення.

Що здається

Посилання на репозиторій `mjsf-mag` із Vue-проєктом у каталозі `lab4` та звіт із титульним аркушем, командами й знімками екранів, що засвідчують працездатність роботи та її виконавця. Звіт завантажується до СЕЗН Moodle у форматі PDF.

Вимоги до змісту та оформлення звіту

- Файл звіту завантажується до СЕЗН Moodle у форматі PDF.
- Звіт містить титульний аркуш і деталі виконання роботи.
- Звіт містить посилання на репозиторій GitHub, Bitbucket або GitLab із роботою в каталозі `lab1`, `lab2`, … за номером роботи.

- Звіт містить команди, які виконуються під час роботи.
- Звіт містить знімки екрана, з яких зрозумілий контекст роботи та видно персональні дані автора: адресний рядок, дані студента й відомості про середовище.

Крім переліченого, за цією роботою здається:

- Вихідний код нового Vue-проєкту в каталозі lab4 з компонентами Tabs, Tab, ModalDialog і панелями для демонстрації локального стану
- package.json і package-lock.json із Sass, конфігурація Tailwind/PostCSS та локальні SCSS-стилі компонентів
- README зі структурою компонентів, командами запуску й збірки, контрактами Tabs і Tab та поясненням перевірки KeepAlive
- Таблиця перевірок з очікуваним і фактичним результатом для трьох варіантів, незалежності наборів, збереження стану, ModalDialog, клавіатури й адаптивності
- Знімок сторінки з трьома варіантами вкладок, парні знімки стану панелі до перемикання та після повернення, DOM із динамічною панеллю і знімки двох перевірочних розмірів екрана
- Результат npm run build і коротке пояснення відмінностей між slot, props, provide/inject, component :is та KeepAlive, а також того, коли панель доцільно задавати розміткою, а коли значенням

Контрольні питання

1. Яке призначення default slot у компоненті Tabs цієї архітектури?
2. Чому ключ tabsKey створюють через Symbol в окремому модулі?
3. Який стандартний контракт v-model має компонент Tabs?
4. Що передають у <component :is="panel">?
5. Чому KeepAlive у вкладках перевіряють разом із v-if на динамічному компоненті, а не лише з v-show?

Література до роботи

1. Vue.js. Slots — https://vuejs.org/guide/components/slots.html
2. Vue.js. Provide / Inject — https://vuejs.org/guide/components/provide-inject.html
3. Vue.js. Dynamic Components — https://vuejs.org/guide/essentials/component-basics.html#dynamic-components
4. Vue.js. KeepAlive — https://vuejs.org/guide/built-ins/keep-alive.html
5. Tailwind CSS 3. Reusing Styles — https://v3.tailwindcss.com/docs/reusing-styles#extracting-classes-with-apply
6. Tailwind CSS 3. Installation with Vite — https://v3.tailwindcss.com/docs/guides/vite#vue
7. Tailwind CSS 3. Responsive Design — https://v3.tailwindcss.com/docs/responsive-design
8. WAI-ARIA APG. Tabs Pattern — https://www.w3.org/WAI/ARIA/apg/patterns/tabs/
9. WAI-ARIA APG. Tabs with Automatic Activation — https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic/
10. W3C. WCAG 2.2. Target Size (Minimum) — https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html
11. W3C. WCAG 2.2. Target Size (Enhanced) — https://www.w3.org/WAI/WCAG22/Understanding/target-size-enhanced.html
12. ISO/IEC/IEEE 31320-1:2012. Syntax and Semantics for IDEF0 — https://www.iso.org/standard/60615.html
13. NIST FIPS 183. Integration Definition for Function Modeling (IDEF0) — https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub183.pdf